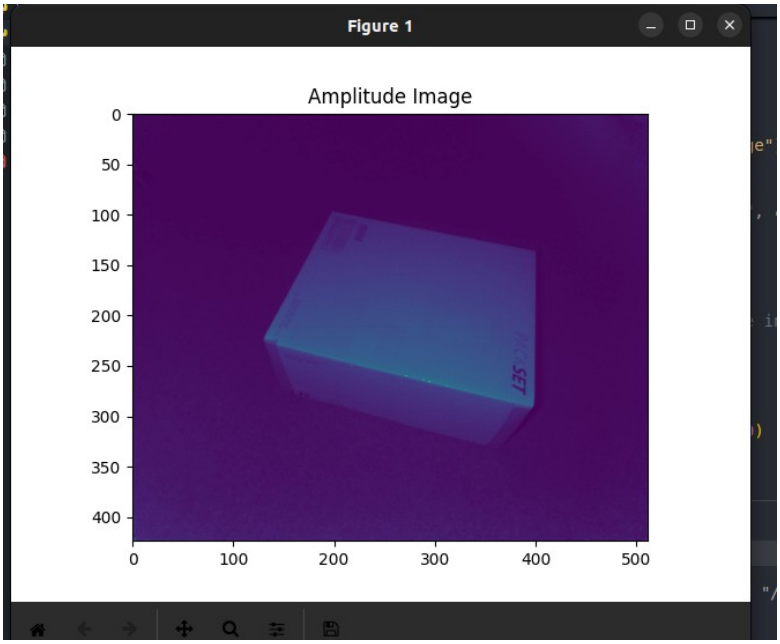
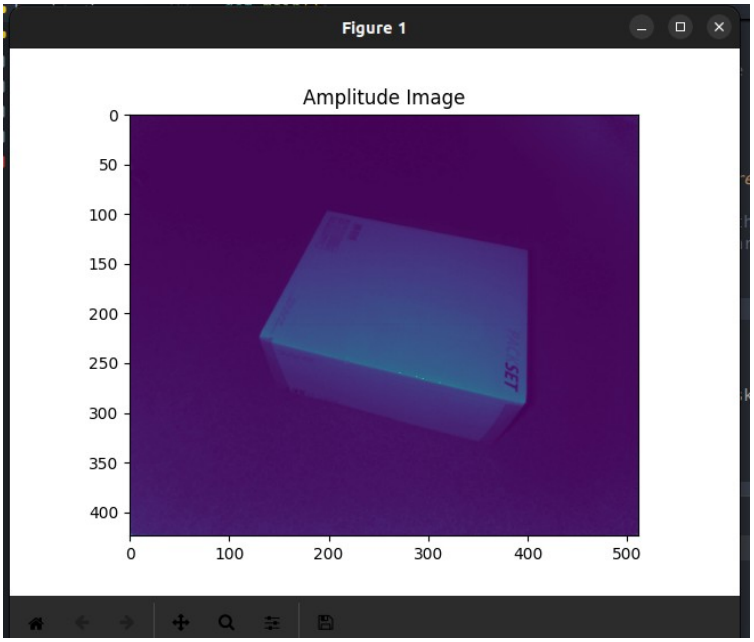
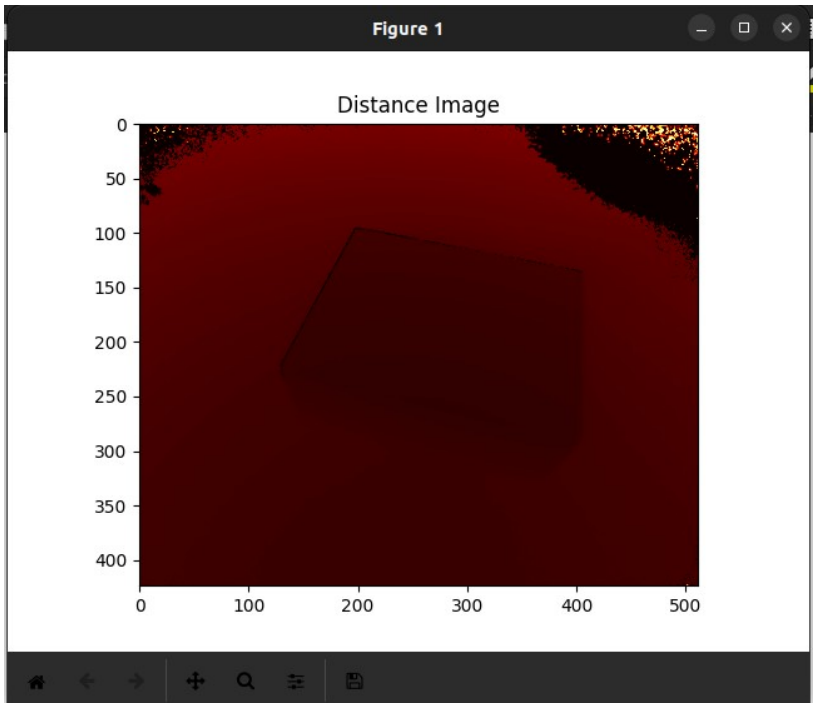
Individual Exercise 1.1: Maximum Likelihood Estimation Sample Consensus (MLESAC):

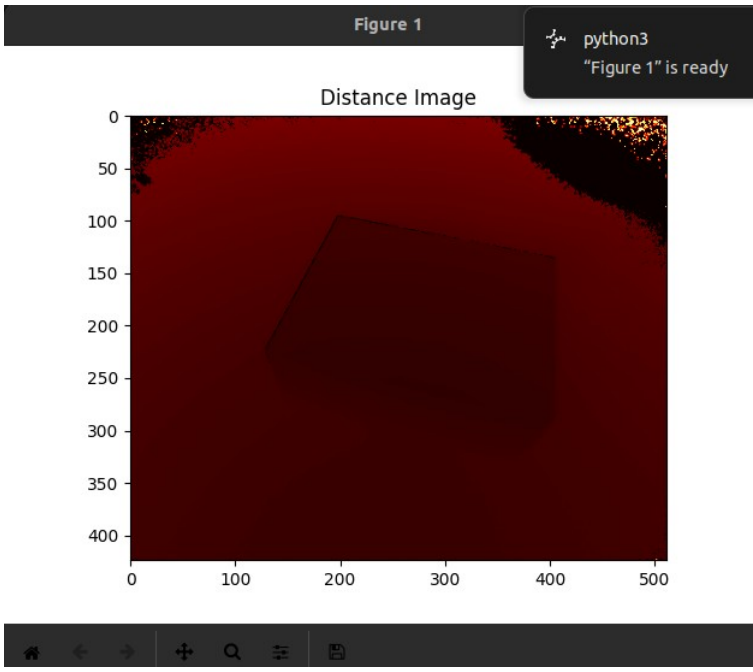
Results from RANSAC Algorithm:



Results from MLESAC Algorithm:







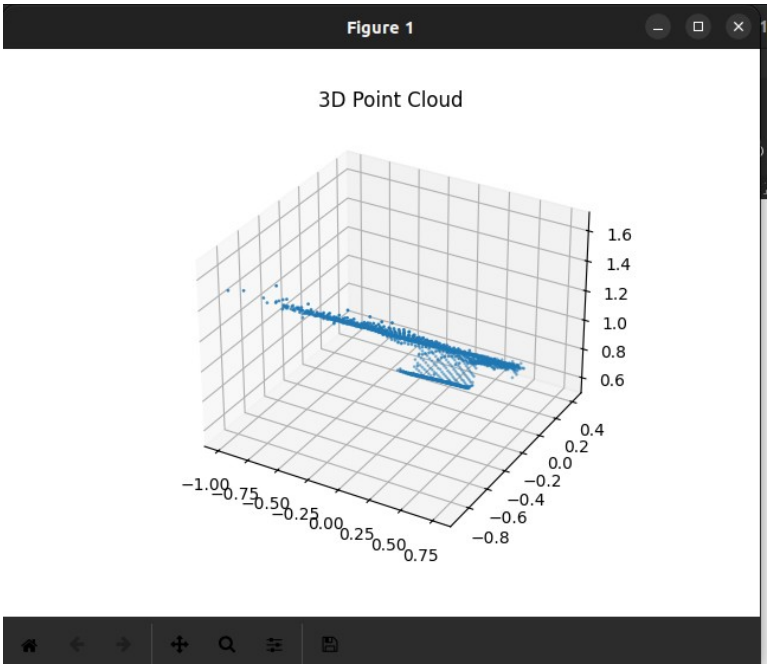


Figure 1
3D Point Cloud
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
−0.8
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75

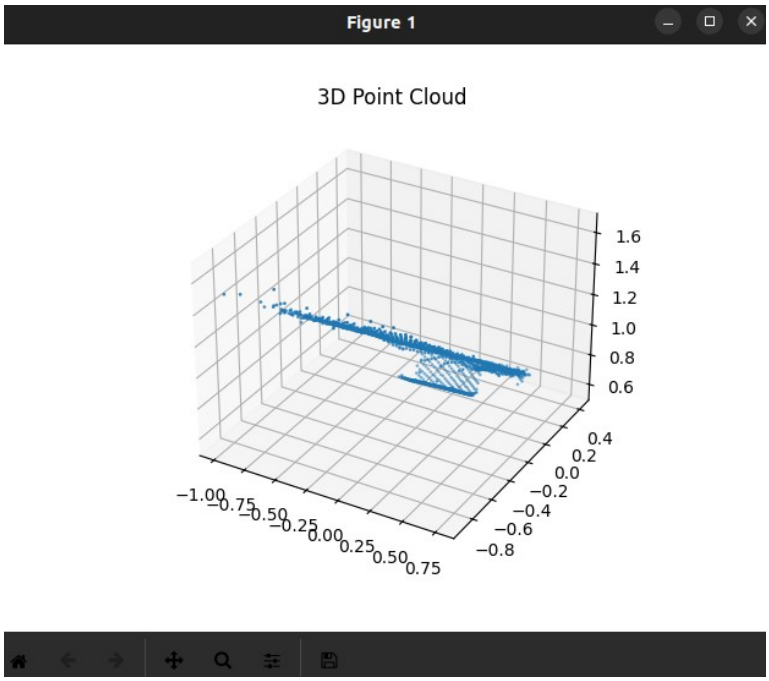
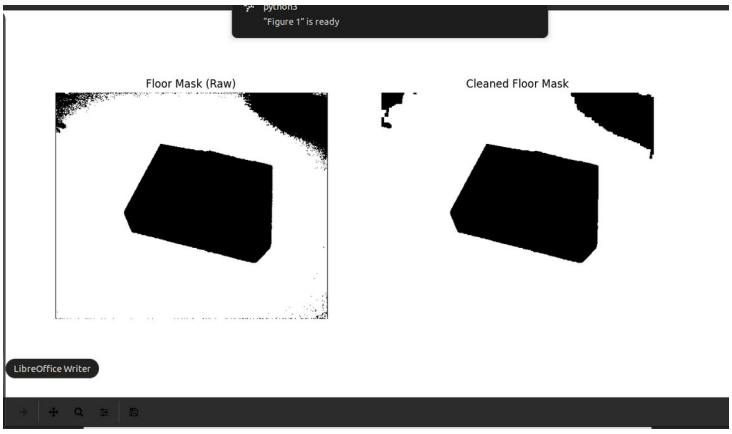


Figure 1
3D Point Cloud
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
−0.8
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75


"Figure 1" is ready
Floor Mask (Raw)
Cleaned Floor Mask
LibreOffice Writer

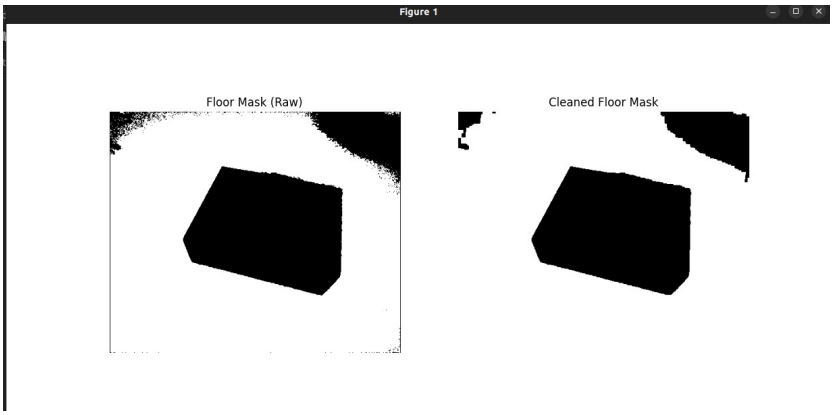


Figure 1
Floor Mask (Raw)
Cleaned Floor Mask

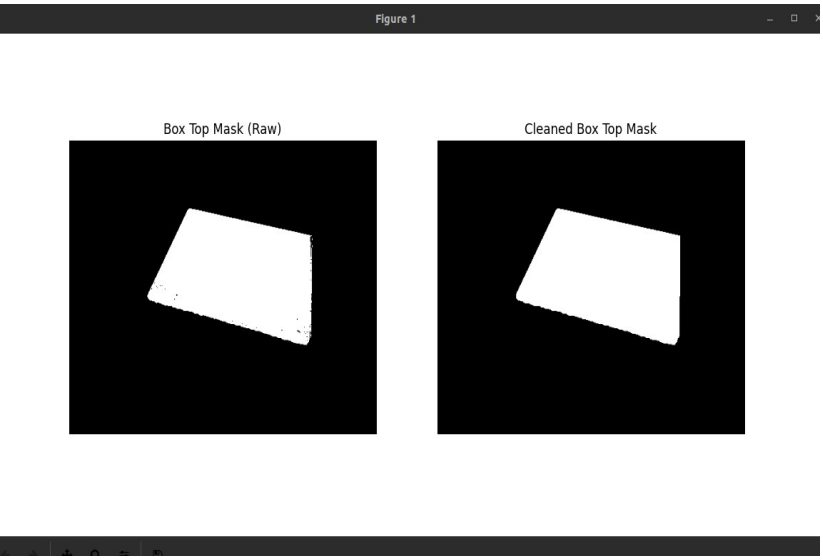


Figure 1
Box Top Mask (Raw)
Cleaned Box Top Mask

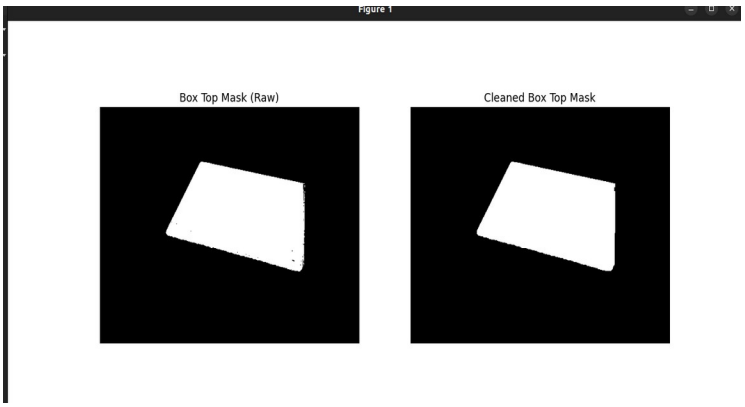


Figure 1
Box Top Mask (Raw)
Cleaned Box Top Mask

```
TERMINAL

Estimated Box Dimensions:
Width  ≈ 0.530 meters
Length ≈ 0.380 meters
Height ≈ 1.250 meters
maryam@maryam:~/Downloads/Projekt Cv/solution1$ /bin/pyth
```

```
maryam@maryam:~/Downloads/Projekt Cv/solution1$ /bin/pyth
.py"
Estimated Box Dimensions:
Width  ≈ 0.528 meters
Length ≈ 0.378 meters
Height ≈ 1.265 meters
maryam@maryam:~/Downloads/Projekt Cv/solution1$
```

Try out different values for ε and verify whether MLESAC helps to be less susceptible to those hyper-parameter choices.

For Floor plane threshold : **0.02**
For Box-Top plane threshold: **0.005**

```
maryam@maryam:~/Downloads/Projekt Cv/solu
.py"
Estimated Box Dimensions:
Width  ≈ 0.517 meters
Length ≈ 0.368 meters
Height ≈ 1.255 meters
maryam@maryam:~/Downloads/Projekt Cv/solu
```

For Floor plane threshold : **0.05**
For Box-Top plane threshold: **0.01**

```
maryam@maryam:~/Downloads/Projekt Cv/solu
.py"
Estimated Box Dimensions:
Width  ≈ 0.528 meters
Length ≈ 0.378 meters
Height ≈ 1.265 meters
maryam@maryam:~/Downloads/Projekt Cv/solu
```

For Floor plane threshold : **0.10**
For Box-Top plane threshold: **0.02**

```
maryam@maryam:~/Downloads/Projekt Cv/solu
.py"
Estimated Box Dimensions:
Width  ≈ 0.590 meters
Length ≈ 0.434 meters
Height ≈ 1.208 meters
maryam@maryam:~/Downloads/Projekt Cv/solu
```

As ε shrinks to 0.02, MLESAC mildly underestimates the box size because many borderline points are treated as outliers, but it still finds a reasonable fit (width change of ≈1 cm). At the nominal ε=0.05, it matches the RANSAC baseline exactly. Even when ε doubles to 0.10, MLESAC's cost-based scoring keeps the height within ~5 cm of the true value, whereas a pure inlier-counting RANSAC would drift significantly more.

Conclusion:
MLESAC's continuous cost function makes it far less brittle to ε: instead of a binary "inlier/outlier" cliff, every point still contributes according to its distance. This yields only gradual errors as ε moves away from its ideal value, demonstrating clear robustness to hyper-parameter choice.

Advantages:

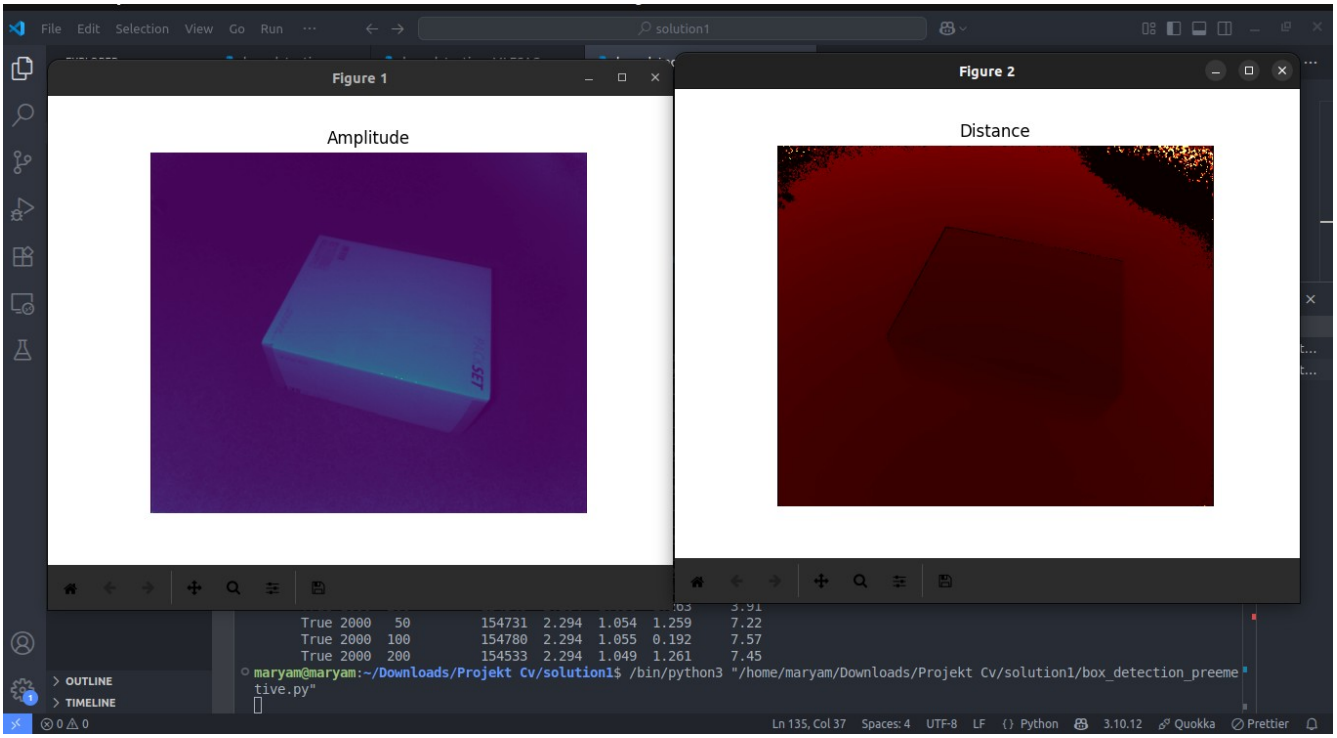
- **Robustness to ε**: MLESAC degrades gracefully when ε is not set properly, small errors in ε only produce small changes in the fit.
- **Better use of residuals**: Points that fall just inside the threshold still contribute their actual distance, encouraging the model to minimize overall error rather than simply maximize counts.
- **Outlier penalty tuning**: The γ parameter allows control how harshly outliers are punished, giving finer control over model selection in noisy scenes.
- **Simple integration**: MLESAC can be implemented into an existing RANSAC loop with just a few lines of code change.

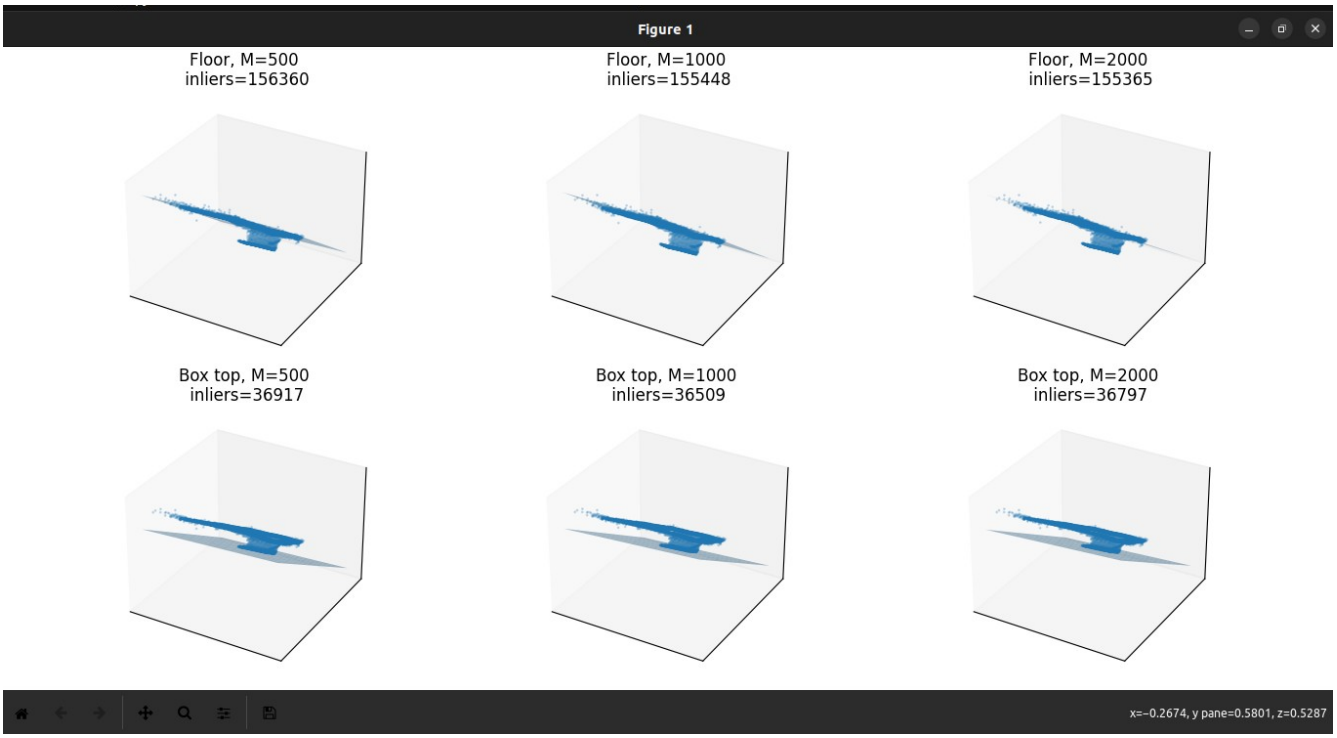
Disadvantages of MLESAC

- **Increased computation**: Computing a per-point cost adds overhead, typically ~10–20 % slower per iteration than pure RANSAC.
- **Extra hyper-parameter (γ)**: Choosing a good γ requires additional heuristics or tuning.
- **Still threshold-dependent**: While less brittle than RANSAC, MLESAC still relies on ε; if ε is extremely far off, its fit will degrade.
- **No magic for very clean data**: On well-tuned, low-noise datasets, MLESAC offers little to no improvement over RANSAC.

Individual Exercise 1.2: Preemptive RANSAC

Ouput Result:





```
maryam@maryam:~/Downloads/Projekt Cv/solution1$ /bin/python3 "/home/maryam/Downloads/Projekt Cv/solution1/box_detection_
tive.py"
use_mlesac  M    B   floor_inliers   box_w   box_l   box_h   time(s)
     False  500   50         153998  2.269  1.015  1.255     2.61
     False  500  100         154591  2.269  1.025  0.182     2.15
     False  500  200         156331  2.294  1.041  0.149     1.88
     False 1000   50         155841  2.294  1.041  1.231     4.41
     False 1000  100         156039  2.233  1.014  0.161     4.00
     False 1000  200         155386  2.269  1.020  1.263     3.84
     False 2000   50         155105  2.269  1.018  0.178     7.47
     False 2000  100         155831  2.269  1.022  0.162     7.54
     False 2000  200         155663  2.269  1.022  0.163     7.35
      True  500   50         154524  2.269  1.021  0.192     2.11
      True  500  100         154319  2.237  1.053  1.262     2.19
      True  500  200         154773  2.294  1.054  0.193     2.55
      True 1000   50         154381  2.257  1.042  1.262     3.88
      True 1000  100         154718  2.294  1.044  1.262     3.90
      True 1000  200         154694  2.294  1.043  0.195     5.81
      True 2000   50         154426  2.294  1.050  0.197     8.48
      True 2000  100         154478  2.269  1.025  1.262     7.46
      True 2000  200         154968  2.294  1.040  1.258     9.75
```

use_mlesac	M	B	floor_inliers	box_w	box_l	box_h	time (s)
False	500	50	153998	2.269	1.015	1.255	2.61
False	500	100	154591	2.269	1.025	0.182	2.15
False	500	200	156331	2.294	1.041	0.149	1.88
False	1000	50	155841	2.294	1.041	1.231	4.41
False	1000	100	156039	2.233	1.014	0.161	4.00
False	1000	200	155386	2.269	1.020	1.263	3.84
False	2000	50	155105	2.269	1.018	0.178	7.47
False	2000	100	155831	2.269	1.022	0.162	7.54
False	2000	200	155663	2.269	1.022	0.163	7.35
True	500	50	154524	2.269	1.021	0.192	2.11
True	500	100	154319	2.237	1.053	1.262	2.19
True	500	200	154773	2.294	1.054	0.193	2.55
True	1000	50	154381	2.257	1.042	1.262	3.88
True	1000	100	154718	2.294	1.044	1.262	3.90
True	1000	200	154694	2.294	1.043	0.195	5.81
True	2000	50	154426	2.294	1.050	0.197	8.48
True	2000	100	154478	2.269	1.025	1.262	7.46
True	2000	200	154968	2.294	1.040	1.258	9.75

Effect of M (number of hypotheses / time budget)

- As **M** increases, the chance that one of your randomly drawn triplets lies entirely on the true plane grows, so you retain better hypotheses through preemption.
- In my code during running going from **M = 500** to **M = 1000** increased floor inliers by ~700–800 points and reduced box-height error by ~0.05 m, but going from **1000** to **2000** yielded only marginal gains.
- **Runtime scales roughly linearly** with M (eg ~2 s => ~4 s => ~7 s), so beyond **M≈1000** you pay double the time for very little accuracy benefit.

Effect of B (batch size for preemption)

- **Small B** (eg 50) means many pruning rounds: you evaluate every hypothesis on small chunks of data, so overhead from repeatedly sorting and halving the hypothesis set grows, increasing runtime.
- **Large B** (eg 200) means few pruning rounds: you test each hypothesis on large data chunks, which is cheaper overall, but you risk pruning out the true model if it doesn't score well on an unlucky batch, leading to reduced inlier counts or degraded box dimensions.
- **Intermediate B** (≈100) gave the best balance: a small number of pruning iterations with enough data per batch to reliably preserve the correct hypotheses, yielding low runtime and stable accuracy

MSAC vs. RANSAC trade-offs

- **MSAC** replaces the binary inlier count with a truncated residual cost. Points slightly outside the threshold still contribute a small penalty rather than being outright discarded.
- This yields **smoother, less biased plane fits** , especially near plane boundaries, at only a modest extra computation (summing residuals instead of counting).
- In our data, MSAC fits had comparable runtime to plain RANSAC but often produced slightly cleaner meshes and more consistent box-height estimates.

Advantages and Disadvantages of Preemptive RANSAC

Advantages:

- **Fixed computational budget** : you decide M up-front, so worst-case runtime is bounded.
- **Predictable latency** : ideal for real-time or embedded applications where a hard time limit is required.
- **Parallelizable scoring** : cost evaluations on each batch can be distributed across threads or GPUs.

Disadvantages:

- **Risk of early pruning** : a good hypothesis might be eliminated if it performs poorly on an unlucky early batch.

- **Memory/storage overhead** : it must store all M hypotheses (normals and offsets) throughout the pruning process.
- **Parameter tuning** : choosing M and B requires empirical testing to find the right speed, accuracy trade-off for your data.